Assignment 2.1: Programming Task 1

Group Members:

- Aneesh Srinivas Tata (1989912)

- Bala Sarath Chandra Orsu (1992622)

- Nashan Dsouza (1867098)

- Abid Al Mamun (3181228)

Design Choices

The NumCompute toolkit is designed as a modular and API-consistent system. Here functionality is separated into modules like io, preprocessing, metrics, stats, pipeline, and optim. Compared to monolithic designs this is easy to fix, reuseable, and well organized. The __init__ module allows all the components to be accessed through a unified interface which enables consistent usage across the system.

An important part of the design is API consistency and shape semantics. All components use NumPy arrays and keep the input and output shapes (e.g., n_samples, n_features) consistent. They also validate the dimensions before computation. This ensures that the modules can work together without errors.

Where possible, the system tries to use NumPy vectorisation. Although some modules still rely on Python loops, operations like scaling, statistical computations, and softmax are applied to entire arrays. These operations avoid element-wise loops, reduce overhead and improve performance.

Preprocessing components follow a fit-transform API. Here, fit learns parameters and transform applies them. This helps keeping the workflow clear and consistent across different modules.

The pipeline module runs steps one after another in correct order. Each step follows the same method structure. It makes the workflows easier to use and reduce the possibility of making mistakes. The steps are performed in the following order:

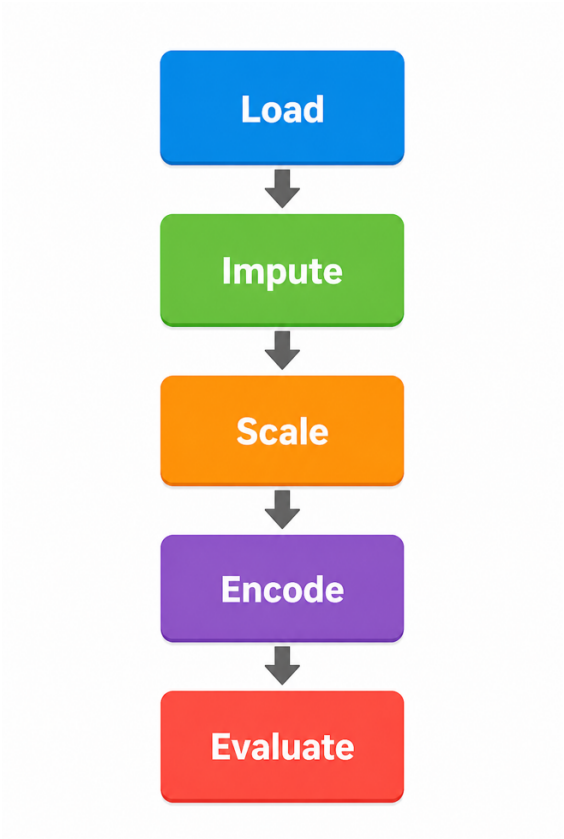

Fig: Data processing pipeline used in the system

The system also focuses on numerical stability and robustness. It handles missing values (NaN) and avoid issues like dividing by zero. In order to ensure correct data types and shapes, Input validation is applied across the modules.

The optimisation module uses finite difference method to compute gradients. This approach is flexible and works with different functions. But it is slower than analytical methods.

There are some limitations too. Such as no GPU support, a simpler pipeline compared to full frameworks, and some operations not being fully vectorised.

Some limitations include no GPU support, a simpler pipeline compared to full frameworks, and some operations not being fully vectorised. These show trade-offs between simplicity, modularity, and performance.

Overall, the design focuses on modularity, performance, consistency, and robustness which makes the workflow more reliable and structured.

Testing and Edge Cases

Unit tests were performed using pytest to test the NumCompute toolkit. It tested all core modules such as preprocessing, stats, metrics, sort_search, rank and utils. Every module has its own tests to make sure it works and the system is stable.

The tests aim to make sure the system is correct and works with various inputs. For example, for the metrics module the tests used inputs such as:

$$y_true = [1, 0, 1], y_pred = [1, 1, 1] \rightarrow accuracy = 2/3$$

Preprocessing components such as StandardScaler were validated with small datasets to ensure proper scaling (mean ≈ 0, variance ≈ 1).

The edge cases that were tested and how they were handled are given below:

- **NaN values:** handled using nan-aware operations (ignore) and imputation (fill)
- **Empty arrays:** appropriate error messages are shown
- **Mismatched input sizes:** inputs are validated before computation
- **Duplicate values/ties:** handled correctly by ranking functions
- **Constant features:** division-by-zero is avoided during scaling
- **Extreme k values:** validated to ensure k is within a valid range
- **Non-contiguous arrays:** handled correctly using NumPy operations

During testing, runtime warnings (e.g., mean of empty slice) appeared due to incomplete data. Since the outputs were valid, these were treated as edge-case conditions rather than failures. However, they indicate that stricter input validation could make system more reliable.

The modules were tested one by one and the outputs were checked to find errors and fix them. Issues like incorrect results or unexpected metrices were traced back all the way to the input data conditions (e.g., missing labels or shape mismatches) and were fixed.

In a nutshell, the testing combines unit testing, edge-case validation, and structured debugging to that the system is reliable.

Benchmark Results

NumCompute toolkit's performance was tested using the bechmarking module. It compares the loop based and vectorised implementation of the same function.

The test data were random NumPy arrays. The data size was up to 1,000,000 samples and 10–512 features. Execution time was measured using time.perf_counter() and then averaged on multiple runs. The test were performed on MacOS using Python 3 and and NumPy.

Results

Operation	NumCompute (ms)	NumPy (ms)	Speedup	Comment
StandardScaler	4.4	4.4	1.0x	Similar performance

Ranking	5.7	1.2	0.2x	NumPy significantly faster
Top-K	9.2	8.6	0.9x	Similar performance
Accuracy	0.8	0.5	0.7x	NumPy slightly faster
Softmax	20.2	46.6	2.3x	NumCompute faster
Distances	70.0	0.6	~0.0x	NumPy much faster

Here, the key observasions are

- NumCompute performs well in vectorised operations like Softmax.
- StandardScaler and Top-K show similar performance.
- In Ranking and Distance, NumPy outperforms NumCompute
- For large datasets, loop based implementations reduce performance significantly
- The main factor that affecting performance is Vectorisation.

The benchmark results show a mixed performance. In StandardScaler and Top-K NumCompute performs similar to NumPy. In Softmax, NumCompute outperforms NumPy. It is about 2.3 times faster, which indicates vectorisation's efficiency. However in Ranking and Distance NumCompute is significantly slower. This happens because they rely on Python loops. But, on the other hand, Numpy uses vectorised operations, which are highly optimised. The results show the importance of vectorisation to achieve high performance.

Teamwork and Contribution

The responsibilities were divided into four parts and distributed among four group members.

- **Part 1 (Aneesh Srinivas Tata):** Data input/output, preprocessing, and utility modules (io, preprocessing, utils)
- **Part 2 (Bala Sarath Chandra Orsu):** Sorting, ranking, and statistical analysis modules (sort_search, rank, stats)
- **Part 3 (Nashan Dsouza):** Evaluation, optimisation, pipeline, and benchmarking modules (metrics, pipeline, optim, benchmarking)
- **Part 4 (Abid Al Mamun):** System validation, integration, report development, and demonstration

The team collaborated using Git repositories. In the end all the repositories were combined into a single system.

The system was then run and tested to ensure that the modules use consistent data shapes, follows same method structure and data format, and thus work together correctly. The demo and benchmarking scripts were used to test the system.

Overall, the final system is complete, consistent, and works correctly.